I3C Controller Design Document v1.0

© Mirafra Technologies

Contents

Contents 1-3

List of Figures 4

List of Tables 5

Chapter 1

Introduction 6-7

1.1 Overview of MIPI I3C Protocol 6
1.2 Purpose of I3C Controller IP 6
1.3 Scope of I3C Controller IP v1.0 6
1.4 Supported Features 7
1.5 Unsupported / Future Features 7
1.6 Compliance with MIPI I3C Specification 7

Chapter 2

Architecture 8-11

2.1 I3C Controller Top-Level Architecture 8
2.2 Block Diagram Description 9
2.3 Data Flow Overview 9
2.4 Control Flow Overview 10
2.5 Clocking and Reset Strategy 10
2.6 Design Assumptions and Constraints 11

Chapter 3

Interface Specification 12-14

3.1 External Signal Interface 12
3.1.1 Clock and Reset Signals 12
3.1.2 I3C Bus Signals (SCL, SDA) 12
3.2 Register Interface 13
3.2.1 Register Access Model 13
3.2.2 Command Programming Interface 13
3.3 FIFO Interfaces 13
3.3.1 TX FIFO Interface 13
3.3.2 RX FIFO Interface 14

Chapter 4

Register Map 15-17

4.1 Register Address Map 15
4.2 Command Registers 15
4.3 Address and CCC Registers 16
4.4 Status Registers 16
4.5 FIFO Control and Status Registers 16
4.6 Register Reset Values 17

Chapter 5

Functional Description 18-21

5.1 Command Execution Flow 18
5.2 SDR Transfer Operation 18

5.2.1 SDR Write Transaction 18
5.2.2 SDR Read Transaction 19
5.3 Common Command Code (CCC) Handling 19
5.3.1 Broadcast CCC 19
5.3.2 Directed CCC 20
5.4 Dynamic Address Assignment (DAA) 20
5.4.1 ENTDAA Sequence 20
5.4.2 PID Reception 20
5.4.3 Dynamic Address Allocation 21
Chapter 6
FSM Architecture 22-27
6.1 SDR FSM 22
6.1.1 State Diagram 23
6.1.2 State Descriptions 23
6.1.3 SDR FSM Timing 24
6.2 DAA FSM 24
6.2.1 State Diagram 25
6.2.2 Arbitration and PID Handling 26
6.3 FSM Synchronization and Arbitration 26
Chapter 7
Bit Engine Design 28-30
7.1 Purpose of Bit Engine 28
7.2 Bit-Level Operations 28
7.2.1 START and STOP Generation 28
7.2.2 Bit Transmit Operation 29
7.2.3 Bit Receive Operation 29
7.2.4 ACK / NACK Handling 29
7.3 Bit Engine Interface Signals 29
7.4 Timing Control and SCL Generation 30
Chapter 8
FIFO Architecture 31-32
8.1 TX FIFO 31
8.1.1 FIFO Depth and Width 31
8.1.2 Write Operation 31
8.1.3 Underflow and Overflow Handling 31
8.2 RX FIFO 32
8.2.1 Data Capture Mechanism 32
8.2.2 Read Operation 32
8.2.3 Status Signaling 32
Chapter 9
Clocking, Reset, and Timing 33-34
9.1 System Clock Requirements 33
9.2 Programmable SCL Frequency 33
9.3 Reset Behavior 33
9.4 Timing Compliance (SDR Mode) 34
Chapter 10

Error Handling and Status Reporting 35-36
10.1 NACK Detection 35
10.2 Address Error Handling 35
10.3 FIFO Error Conditions 36
10.4 Command Error Reporting 36
Chapter 11
RTL Implementation Details 37-39
11.1 Module Hierarchy 37
11.2 Parameterization 38
11.3 Coding Guidelines 38
11.4 Synthesizability Considerations 39
Chapter 12
Verification Readiness 40-41
12.1 Design for Verification (DFV) 40
12.2 Assertion Strategy 40
12.3 RTL Observability and Debug Signals 40
Chapter 13
Performance and Limitations 42-43
13.1 Maximum Supported Transfer Size 42
13.2 Throughput Considerations 42
13.3 Known Limitations (v1.0) 42
Chapter 14
Future Enhancements 44-45
14.1 HDR Mode Support 44
14.2 IBI Support 44
14.3 Slave Mode Support 44
14.4 Multi-Controller Support 45
Chapter 15
Conclusion 46
Appendix A
Signal List 47-50
Appendix B
FSM State Tables 51
Appendix C
Timing Diagrams 52-53
Appendix D
References 54

List of Figures

Fig no	Name of the Figure	Pg no
Figure 2.1	I3C Controller Top-Level Architecture	8
Figure 6.1	Flow chart of SDR FSM	22
Figure 6.2	Flow chart of DAA FSM	25
Figure C.1	SDR Address Phase and Write Transaction Timing Diagram	52
Figure C.2	SDR Read Transaction Timing Diagram	52
Figure C.3	DAA Arbitration and Dynamic Address Assignment with Device Table Update	53

List of Tables

Table No.	Name of the Table	Pg no
Table A.1	Top-Level Module: I3C_TOP	47
Table A.1.2	CPU / Register Interface	47
Table A.1.3	I3C Bus Interface Signals	47
Table A.2.1	Control and Status Signals	47
Table A.3.1	TX FIFO (i3c_Tx_FIFO)	48
Table A.3.2	RX FIFO (i3c_Rx_FIFO)	48
Table A.4	SDR FSM Module: i3c_sdr_fsm	49
Table A.5	DAA FSM Module: i3c_daa_fsm	49
Table A.6	Bit Engine Module: i3c_bit_engine	50
Table A.7	Device Table Module: i3c_device_table	50
Table A.8	SCL Generator Module: i3c_scl_gen	50
Table B.1	SDR FSM State Table	51
Table B.2:	DAA FSM State Table	51

Chapter 1

Introduction

1.1 Overview of MIPI I3C Protocol

The MIPI Improved Inter Integrated Circuit (I3C) protocol is a next-generation serial interface standard developed by the MIPI Alliance. It is designed to overcome the limitations of legacy I2C while maintaining backward compatibility with existing I2C devices. I3C provides higher data rates, improved power efficiency, dynamic addressing, and enhanced protocol features suitable for modern SoCs and sensor-based systems.

Unlike I2C, which relies on fixed addressing and relatively low-speed operation, I3C introduces Dynamic Address Assignment (DAA), Common Command Codes (CCCs), and higher-speed Single Data Rate (SDR) transfers. These enhancements make I3C well-suited for applications requiring scalable sensor connectivity, low latency, and reduced pin count.

The protocol supports a centralized controller (master) managing one or more target devices over a two-wire bus consisting of Serial Clock (SCL) and Serial Data (SDA) signals. I3C defines multiple operating modes, including SDR and High Data Rate (HDR) modes, along with optional advanced features such as In-Band Interrupts (IBI) and Hot-Join.

1.2 Purpose of I3C Controller IP

The purpose of the I3C Controller IP is to provide a synthesizable, configurable, and reusable RTL implementation of an I3C master capable of managing I3C bus transactions in compliance with the MIPI I3C specification. This IP abstracts the low-level bus timing and protocol complexities from the system software, enabling seamless integration into SoC designs.

The controller IP is intended to:

- Act as the primary bus controller for I3C-compliant target devices
- Manage address assignment and bus ownership
- Execute programmed I3C transactions such as private transfers and CCC operations
- Provide a register-based programming model suitable for firmware and drivers
- Interface cleanly with verification environments and system-level testbenches

The IP is designed with modularity and extensibility in mind, allowing future enhancement without major architectural changes.

1.3 Scope of I3C Controller IP v1.0

I3C Controller IP Version 1.0 focuses on implementing a stable and functional subset of the I3C protocol, targeting early integration, validation, and bring-up use cases. The scope of this version is deliberately constrained to reduce complexity while ensuring correctness and compliance for supported features.

Version 1.0 operates exclusively in SDR mode and supports essential controller-side functionalities such as dynamic address assignment, basic Common Command Codes, and private read/write transfers. The IP functions solely as an I3C controller (master) and does not include target (slave) functionality.

This version serves as a foundational implementation upon which advanced protocol features and performance optimizations can be built in future releases.

1.4 Supported Features

The following features are supported in I3C Controller IP v1.0:

- I3C Controller (Master) operation
- Single Data Rate (SDR) mode transfers
- Private I3C Read and Write transactions
- Dynamic Address Assignment using ENTDAA CCC
- Support for Broadcast and Directed Common Command Codes
- Programmable SCL frequency generation
- FIFO-based transmit and receive data buffering
- Register-based command and status interface
- Modular FSM-based architecture (SDR FSM, DAA FSM)
- Centralized Bit Engine for protocol timing and signal control

1.5 Unsupported / Future Features

The following I3C features are not supported in Version 1.0 and are planned for future enhancements:

- High Data Rate (HDR) modes (DDR, Ternary, etc.)
- In-Band Interrupt (IBI) support
- Target (Slave) mode operation
- Hot-Join support
- Multi-controller arbitration
- Advanced power management features

These features are intentionally excluded from v1.0 to keep the initial implementation focused, verifiable, and robust.

1.6 Compliance with MIPI I3C Specification

The I3C Controller IP v1.0 is architected to be compliant with the MIPI I3C Basic Specification for all supported features and operating modes. Protocol behaviour, bus signalling, and transaction sequencing adhere to the requirements defined for SDR controller operation. Any deviations or unimplemented optional features are explicitly documented in this specification. The design ensures that unsupported features are safely ignored or prevented through configuration, avoiding undefined bus behaviour. Compliance validation is supported through structured verification, including directed test scenarios for SDR transfers, CCC execution, and Dynamic Address Assignment flows.

Chapter 2

Architecture

2.1 I3C Controller Top-Level Architecture

The I3C Controller IP is designed as a modular, layered architecture that separates protocol control, transaction sequencing, and bit-level signalling. This separation improves readability, verification, and future extensibility of the design.

At the top level, the controller operates as a single I3C bus controller (master) responsible for initiating and managing all bus transactions. The architecture is centered around a command-driven execution model, where software programs transactions through registers, and the hardware autonomously executes the corresponding I3C protocol sequences.

The major architectural layers include:

- Control Layer: Command decoding, arbitration between transaction types, and high-level sequencing
- Protocol FSM Layer: SDR FSM and DAA FSM handling protocol-specific transaction flows
- Bit-Level Layer: A centralized Bit Engine responsible for SCL/SDA control and timing
- Data Buffering Layer: TX and RX FIFOs for decoupling bus timing from system access
- Configuration Layer: Register interface for software control and status reporting

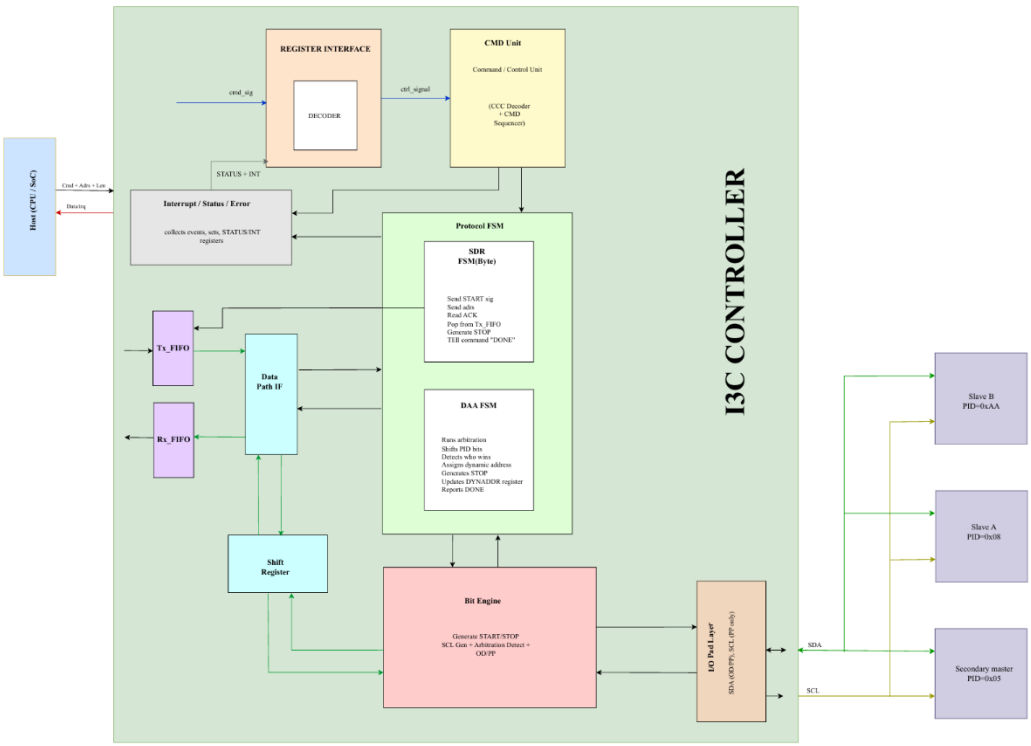

Figure 2.1: I3C Controller Top-Level Architecture

This layered approach ensures clear ownership of responsibilities across modules and simplifies verification and debug.

2.2 Block Diagram Description

The I3C Controller IP consists of the following major blocks:

- **Register Interface Block**
  Provides a memory-mapped programming interface for software to configure commands, addresses, CCCs, transfer length, and clock parameters. It also exposes status and interrupt-related information.
- **Command Control Block**
  Decodes programmed commands and dispatches them to the appropriate protocol FSM (SDR FSM or DAA FSM). It ensures that only one transaction is active at a time and manages command completion signaling.
- **SDR FSM**
  Handles all SDR-based private read/write transactions and CCC transfers. It sequences START, address, data, ACK/NACK, and STOP conditions in compliance with the I3C SDR protocol.
- **DAA FSM**
  Implements the Dynamic Address Assignment flow using the ENTDAA Common Command Code. It manages arbitration, PID reception, and assignment of dynamic addresses to target devices.
- **Bit Engine**
  Acts as the lowest-level protocol block, generating SCL timing and driving or sampling SDA. All FSMs interact with the Bit Engine through well-defined control and status signals.
- **TX FIFO**
  Buffers data to be transmitted on the I3C bus, decoupling system-side writes from bus timing constraints.
- **RX FIFO**
  Buffers data received from the I3C bus, allowing system software to read data asynchronously.

Together, these blocks form a cohesive controller capable of executing complete I3C transactions with minimal software intervention.

2.3 Data Flow Overview

The data flow within the I3C Controller IP follows a unidirectional buffering model for both transmit and receive paths.

Transmit Data Flow:

1. System software writes data into the TX FIFO via the register interface.
2. During an active transaction, the SDR FSM requests data bytes from the TX FIFO.
3. Data bytes are forwarded to the Bit Engine for serial transmission on the SDA line.
4. FIFO status signals indicate empty or near-empty conditions to prevent underrun.

Receive Data Flow:

1. Incoming data bits from the SDA line are sampled by the Bit Engine.
2. Assembled data bytes are passed to the SDR FSM.
3. The SDR FSM pushes received bytes into the RX FIFO.
4. System software reads data from the RX FIFO via the register interface.

This FIFO-based data flow isolates bus timing from system access latency and improves robustness under varying system conditions.

2.4 Control Flow Overview

The control flow is driven by a command-based execution model.

1. Software programs command registers with transaction type, address, length, and optional CCC information.
2. A command start signal triggers the Command Control block.
3. Command Control selects the appropriate FSM (SDR FSM or DAA FSM) based on command type.
4. The selected FSM sequences the transaction by issuing requests to the Bit Engine.
5. The Bit Engine performs the requested bit-level operations and reports completion or error status.
6. Upon transaction completion, status is updated and the controller returns to the idle state.

Only one FSM is active at any given time, ensuring deterministic bus behavior and simplifying arbitration logic.

2.5 Clocking and Reset Strategy

The I3C Controller IP operates using a single system clock domain. All internal logic, including FSMs, FIFOs, and the Bit Engine, is synchronous to this clock.

- **System Clock**
  A free-running clock used for internal state machines, counters, and FIFO logic.
- **SCL Generation**
  The Bit Engine derives the I3C SCL signal from the system clock using programmable clock dividers, enabling software-controlled SCL frequency configuration.
- **Reset Strategy**
  A global active-low reset initializes all registers, FSMs, FIFOs, and internal control logic to a known idle state. Upon reset deassertion, the controller remains idle until explicitly programmed by software.

The reset mechanism ensures that no unintended bus activity occurs during initialization or recovery from error conditions.

2.6 Design Assumptions and Constraints

The design of I3C Controller IP v1.0 is based on the following assumptions and constraints:

- Single I3C controller (no multi-controller arbitration)
- Controller-only (master-only) operation
- SDR mode operation only
- Clean, single-clock-domain design
- Target devices are compliant with the MIPI I3C specification
- Software ensures valid command programming sequences
- Unsupported features are disabled or ignored safely

These assumptions allow the architecture to remain simple, deterministic, and verifiable, forming a strong foundation for future feature expansion.

Chapter 3

Interface Specification

This chapter describes the external and internal interfaces of the I3C Controller IP. The interfaces define how the controller interacts with the system, software, FIFOs, and the I3C bus. All signals and interfaces are designed to be synchronous to a single system clock unless explicitly stated otherwise.

3.1 External Signal Interface

The external signal interface consists of clocking, reset, and I3C bus signals. These signals connect the I3C Controller IP to the SoC infrastructure and the physical I3C bus.

3.1.1 Clock and Reset Signals

The I3C Controller IP operates entirely within a single clock domain.

- **System Clock (clk)**
  This clock drives all internal logic, including FSMs, FIFOs, counters, and the Bit Engine. The clock frequency is assumed to be sufficiently higher than the maximum programmed I3C SCL frequency to allow accurate timing generation.
- **Global Reset (rst_n)**
  An active-low synchronous reset initializes the controller to a known idle state. On reset assertion:
  - All FSMs transition to their IDLE states
  - FIFOs are flushed and marked empty
  - Control and status registers are reset to default values
  - No activity is driven on the I3C bus

After reset deassertion, the controller remains idle until explicitly programmed via the register interface.

3.1.2 I3C Bus Signals (SCL, SDA)

The I3C Controller IP interfaces with the I3C bus using two signals:

- **Serial Clock (SCL)**
  The SCL signal is generated by the controller through the Bit Engine. The frequency of SCL is programmable via configuration registers. SCL timing conforms to SDR mode requirements of the MIPI I3C specification.
- **Serial Data (SDA)**
  The SDA signal is bidirectional and open-drain in nature. The controller drives SDA during write operations and releases it during read operations and arbitration phases. SDA is sampled by the Bit Engine during read and ACK/NACK phases.

The Bit Engine ensures proper control of SDA direction and timing to avoid bus contention.

3.2 Register Interface

The register interface provides a software-visible programming model that allows firmware to configure, initiate, and monitor I3C transactions. All registers are accessed synchronously to the system clock.

3.2.1 Register Access Model

The register interface follows a memory-mapped register model suitable for standard SoC interconnects.

- Registers are accessed using read and write operations
- Control registers configure transaction parameters and controller behavior
- Status registers provide real-time and latched status information
- FIFO access registers allow data movement between software and FIFOs

All register writes are sampled synchronously, and register reads return stable values corresponding to the current controller state.

3.2.2 Command Programming Interface

The command programming interface enables software to initiate I3C transactions using a command-based model.

A typical command sequence includes:

1. Programming transaction parameters such as target address, transfer length, direction, and CCC information
2. Writing data into the TX FIFO (for write transactions)
3. Asserting a command start control bit
4. Monitoring status registers for command completion or error conditions

Once a command is started, the controller autonomously executes the transaction and updates status registers upon completion. Software intervention is not required during active bus transactions.

3.3 FIFO Interfaces

The I3C Controller IP uses FIFO-based buffering to decouple system-side access timing from I3C bus timing. Separate FIFOs are used for transmit and receive paths.

3.3.1 TX FIFO Interface

The TX FIFO stores data bytes to be transmitted on the I3C bus.

- Data is written into the TX FIFO by software via the register interface
- The SDR FSM reads data from the TX FIFO during active write transactions
- FIFO status signals indicate empty and full conditions
- Underflow protection prevents invalid data transmission

The TX FIFO ensures continuous data availability during I3C write operations.

3.3.2 RX FIFO Interface

The RX FIFO stores data bytes received from the I3C bus.

- Data bytes are written into the RX FIFO by the SDR FSM
- Software reads received data via the register interface
- FIFO status signals indicate empty and full conditions
- Overflow protection ensures data integrity

The RX FIFO allows software to read received data asynchronously without stalling bus activity.

Chapter 4

Register Map

This chapter describes the register programming model of the I3C Controller IP. The register set provides software-visible control and status for configuring transactions, initiating commands, monitoring execution progress, and accessing FIFO data.

All registers are memory-mapped and synchronous to the system clock. Unless otherwise stated, registers are cleared to their reset values upon assertion of the global reset.

4.1 Register Address Map

The register address map defines the layout of control, status, and data registers within the I3C Controller IP address space.

The register space is logically divided into the following regions:

- Command and control registers
- Address and CCC configuration registers
- Status and error reporting registers
- FIFO access and FIFO status registers

Each register occupies a fixed address offset and supports word-aligned access. Unused or reserved address locations return zero on reads and ignore writes.

4.2 Command Registers

Command registers are used to configure and initiate I3C transactions. These registers are typically programmed by software before asserting a command start.

Key command-related registers include:

- **Command Control Register**
  Used to initiate a transaction. Writing a command start bit triggers the Command Control logic to dispatch the transaction to the appropriate FSM.
- **Command Type Register**
  Specifies the type of transaction to be executed, such as:
  - Private Write
  - Private Read
  - CCC Operation
  - Dynamic Address Assignment (DAA)
- **Transfer Length Register**
  Defines the number of data bytes to be transferred during the transaction. The length applies to both read and write operations.

Once a command is started, command registers are typically locked until the transaction completes to prevent mid-transaction reconfiguration.

4.3 Address and CCC Registers

Address and CCC registers define protocol-specific parameters required for transaction execution.

- **Target Address Register**
  Holds the 7-bit dynamic address of the target device for private transfers and directed CCCs.
- **CCC Code Register**
  Specifies the Common Command Code to be issued during CCC transactions.
- **CCC Direction Control**
  Indicates whether the CCC operation is a read or write transaction.

These registers are sampled by the controller at the start of command execution and remain constant throughout the transaction.

4.4 Status Registers

Status registers provide visibility into the operational state of the controller and the outcome of executed commands.

Typical status information includes:

- Controller busy or idle indication
- Command completion status
- Error flags such as NACK detection or protocol violation
- FIFO status summary

Status bits may be either real-time or latched until explicitly cleared by software, depending on the type of condition being reported.

4.5 FIFO Control and Status Registers

FIFO-related registers provide access to transmit and receive data buffers as well as FIFO status.

- **TX FIFO Data Register**
  Used by software to write data bytes into the TX FIFO for transmission.
- **RX FIFO Data Register**
  Used by software to read data bytes received from the I3C bus.
- **FIFO Status Registers**
  Provide information such as:
  - TX FIFO empty or full
  - RX FIFO empty or full
  - FIFO overflow or underflow conditions

These registers allow software to manage data flow without directly interacting with bus timing.

4.6 Register Reset Values

Each register has a defined reset value to ensure deterministic controller behavior after reset.

On reset assertion:

- Control registers are cleared to disable all transactions
- Status registers reflect an idle, error-free state
- FIFO contents are flushed
- FIFO pointers are reset

Default reset values are chosen to ensure that no unintended I3C bus activity occurs until software explicitly programs and starts a command.

Chapter 5

Functional Description

This chapter describes the functional behavior of the I3C Controller IP during command execution and protocol operation. It explains how software-programmed commands are translated into I3C bus transactions, covering SDR transfers, Common Command Code (CCC) handling, and Dynamic Address Assignment (DAA).

5.1 Command Execution Flow

The I3C Controller IP follows a command-driven execution model. Software programs all required transaction parameters through registers before initiating execution.

The command execution flow proceeds as follows:

1. Software configures the command registers, including command type, target address, transfer length, and optional CCC information.
2. Data bytes are written into the TX FIFO if the transaction involves a write operation.
3. Software asserts the command start control bit.
4. The Command Control block validates the programmed command and selects the appropriate protocol FSM.
5. The selected FSM (SDR FSM or DAA FSM) sequences the transaction autonomously.
6. Upon completion or error, status registers are updated and the controller returns to the idle state.

During command execution, the controller reports a busy status and prevents concurrent command initiation. This ensures deterministic bus operation and prevents protocol conflicts.

5.2 SDR Transfer Operation

SDR (Single Data Rate) mode is the primary data transfer mode supported in I3C Controller IP v1.0. SDR transfers support both private write and private read transactions between the controller and a target device.

All SDR transfers follow the standard I3C transaction structure:

- START condition
- Address phase
- Data phase
- STOP condition

5.2.1 SDR Write Transaction

In an SDR write transaction, the controller transmits data bytes to a target device.

The SDR write sequence is as follows:

1. The controller issues a START condition on the I3C bus.
2. The 7-bit target address and write direction bit are transmitted.
3. The controller waits for an ACK from the target.
4. Data bytes are fetched sequentially from the TX FIFO and transmitted on the bus.
5. After each byte, the controller samples the ACK/NACK response.
6. If all bytes are successfully transmitted, the controller issues a STOP condition.
7. Status registers are updated to indicate successful completion.

If a NACK is detected at any stage, the controller terminates the transaction and reports an error condition.

5.2.2 SDR Read Transaction

In an SDR read transaction, the controller receives data bytes from a target device.

The SDR read sequence is as follows:

1. The controller issues a START condition.
2. The 7-bit target address and read direction bit are transmitted.
3. Upon receiving an ACK, the controller releases SDA and begins sampling incoming data.
4. Data bytes are received and pushed into the RX FIFO.
5. After each byte, the controller drives an ACK or NACK based on whether more data is expected.
6. After the final byte, the controller issues a STOP condition.
7. Status registers are updated to indicate completion.

The RX FIFO allows received data to be read asynchronously by software after transaction completion.

5.3 Common Command Code (CCC) Handling

Common Command Codes (CCCs) are special I3C commands used for bus management and configuration. The I3C Controller IP supports both broadcast and directed CCC operations.

CCC transactions are executed using the SDR FSM with CCC-specific sequencing.

5.3.1 Broadcast CCC

Broadcast CCCs are addressed to all I3C target devices on the bus.

The broadcast CCC flow includes:

1. START condition
2. Broadcast address transmission
3. CCC code transmission
4. Optional data phase (for write CCCs)
5. STOP condition

Broadcast CCCs do not expect data responses from individual targets. The controller does not sample ACKs from specific devices beyond protocol-required signalling.

5.3.2 Directed CCC

Directed CCCs are addressed to a specific target device.

The directed CCC flow includes:

1. START condition
2. Target address transmission
3. CCC code transmission
4. Optional data phase or readback phase
5. STOP condition

Directed CCCs may involve read or write operations, depending on the CCC type. The controller handles ACK/NACK responses and data transfers accordingly.

5.4 Dynamic Address Assignment (DAA)

Dynamic Address Assignment allows the controller to assign unique dynamic addresses to I3C target devices at runtime. This process is initiated using the ENTDAA CCC and is managed by a dedicated DAA FSM.

5.4.1 ENTDAA Sequence

The ENTDAA sequence begins with the controller issuing the ENTDAA broadcast CCC. This places all unaddressed I3C targets into arbitration mode.

During arbitration:

- Target devices transmit their Provisional IDs (PIDs)
- Arbitration ensures that only one device responds at a time

The controller remains in control of SCL and monitors SDA for arbitration outcomes.

5.4.2 PID Reception

The controller receives the PID from the winning target device bit by bit via the Bit Engine.

PID reception includes:

- Sampling manufacturer ID bits
- Sampling part ID and instance ID fields
- Verifying PID integrity

The received PID may be stored internally or forwarded to software for device identification.

5.4.3 Dynamic Address Allocation

After successful PID reception, the controller assigns a dynamic address to the target device.

The allocation flow includes:

1. Transmitting the assigned dynamic address
2. Waiting for target acknowledgment
3. Recording successful assignment
4. Repeating the ENTDAA process for remaining unaddressed devices

The DAA process completes when no additional devices respond to the ENTDAA command. Upon completion, the controller returns to the idle state and updates status registers accordingly.

Chapter 6

FSM Architecture

This chapter describes the finite state machine (FSM) architecture used to control I3C protocol execution. The I3C Controller IP employs dedicated FSMs for SDR transactions and Dynamic Address Assignment (DAA), ensuring clean separation of protocol responsibilities and deterministic bus behavior.

6.1 SDR FSM

The SDR FSM controls all Single Data Rate (SDR) based I3C transactions, including private read/write transfers and Common Command Code (CCC) operations. It is responsible for sequencing protocol phases while delegating bit-level timing and signaling to the Bit Engine.

The SDR FSM operates synchronously with the system clock and transitions between states based on command inputs, Bit Engine handshakes, and protocol responses.

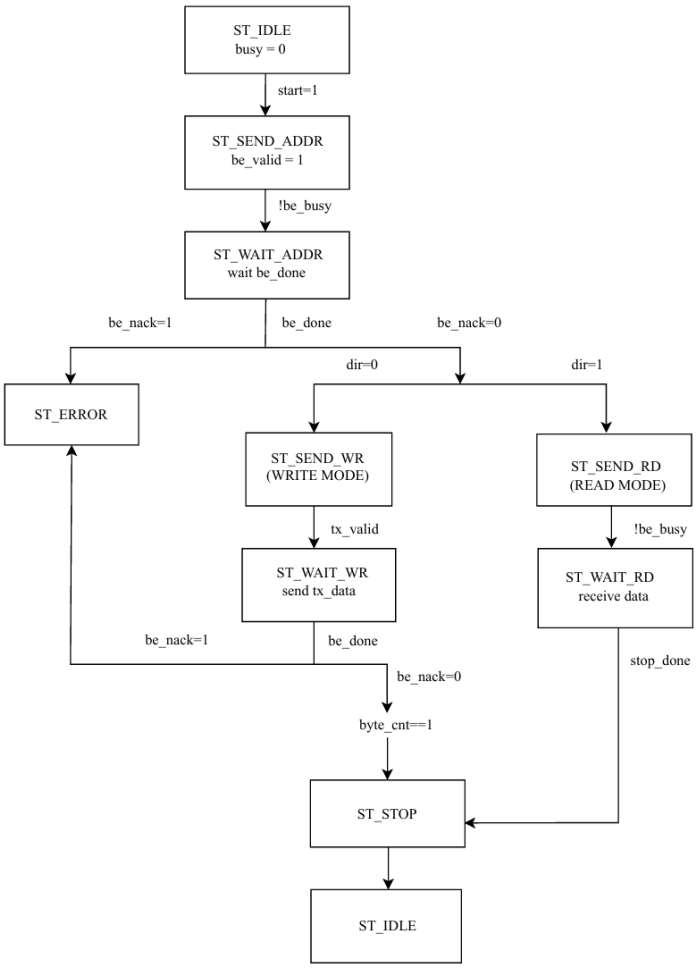

Figure 6.1: Flow chart of SDR FSM

6.1.1 State Diagram

The SDR FSM implements the I3C SDR transaction flow using a compact and deterministic state machine.
Based on the implemented RTL, the FSM consists of the following states:

- IDLE
- SEND_ADDR
- WAIT_ADDR
- SEND_WR
- WAIT_WR
- SEND_RD
- WAIT_RD
- STOP
- ERROR

The FSM progresses through address, data, and stop phases, with conditional branching based on transfer direction (read/write) and runtime events such as ACK/NACK detection and FIFO readiness.

6.1.2 State Descriptions

IDLE
This is the default state after reset and after completion of any transaction. The FSM waits for a start request from the Command Control logic. The busy signal is deasserted in this state.

SEND_ADDR
In this state, the FSM initiates address transmission by asserting be_valid and providing the concatenated {address, direction} byte to the Bit Engine. The FSM remains in this state until the Bit Engine is ready to accept the request.

WAIT_ADDR
The FSM waits for completion of the address phase as indicated by be_done. If a NACK (be_nack) is detected, the FSM transitions to the ERROR state. If an ACK is received, the FSM branches to either write or read flow based on the dir signal.

SEND_WR
This state is used for write transactions. The FSM waits for valid data from the TX FIFO (tx_valid). Once valid data is available, the FSM asserts tx_ready and initiates a write transfer via the Bit Engine.

WAIT_WR
The FSM waits for completion of the write byte transfer. If a NACK is detected, the FSM transitions to ERROR. If the transfer is successful:

- If the current byte is the last byte (byte_cnt == 1), the FSM transitions to STOP
- Otherwise, it loops back to SEND_WR to transmit the next byte

SEND_RD
This state initiates a read operation by requesting the Bit Engine to receive a data byte from the bus. The FSM transitions to WAIT_RD once the Bit Engine is ready.

WAIT_RD
The FSM waits for read data completion. Upon successful reception:

- The received byte is captured and forwarded to the RX FIFO
- The byte counter is decremented

If RX FIFO is not ready when the transfer completes, the FSM transitions to ERROR. If the last byte has been received, the FSM proceeds to STOP; otherwise, it loops back to SEND_RD.

STOP
The FSM completes the transaction by issuing a STOP condition. The done signal is asserted for one cycle, and the FSM returns to IDLE.

ERROR
This state is entered upon protocol or flow-control errors such as:

- Address or data NACK
- RX FIFO not ready

The FSM asserts the error and done signals and safely terminates the transaction before returning to IDLE.

6.1.3 SDR FSM Timing

The SDR FSM operates in close coordination with the Bit Engine using a request–completion handshake.

Key timing characteristics include:

- FSM state transitions occur only after Bit Engine completion signals (be_done, stop_done)
- SCL generation and timing are fully controlled by the Bit Engine
- SDA setup and hold requirements are enforced by Bit Engine timing logic
- FIFO read and write operations occur only at byte boundaries
- Push-pull mode is enabled during I3C write phases and disabled during STOP and IDLE states

This architecture ensures protocol-compliant SDR timing independent of system clock frequency.

6.2 DAA FSM

The DAA FSM manages the Dynamic Address Assignment (DAA) procedure using the ENTDAA Common Command Code. It implements a controller-side, SDR-based DAA flow

compliant with the MIPI I3C specification. The FSM operates independently of the SDR FSM and is activated only when a DAA command is issued by the Command Control logic.

The implementation focuses on reliable address assignment and arbitration handling while intentionally excluding optional or advanced behaviors such as Hot-Join handling in Version 1.0.

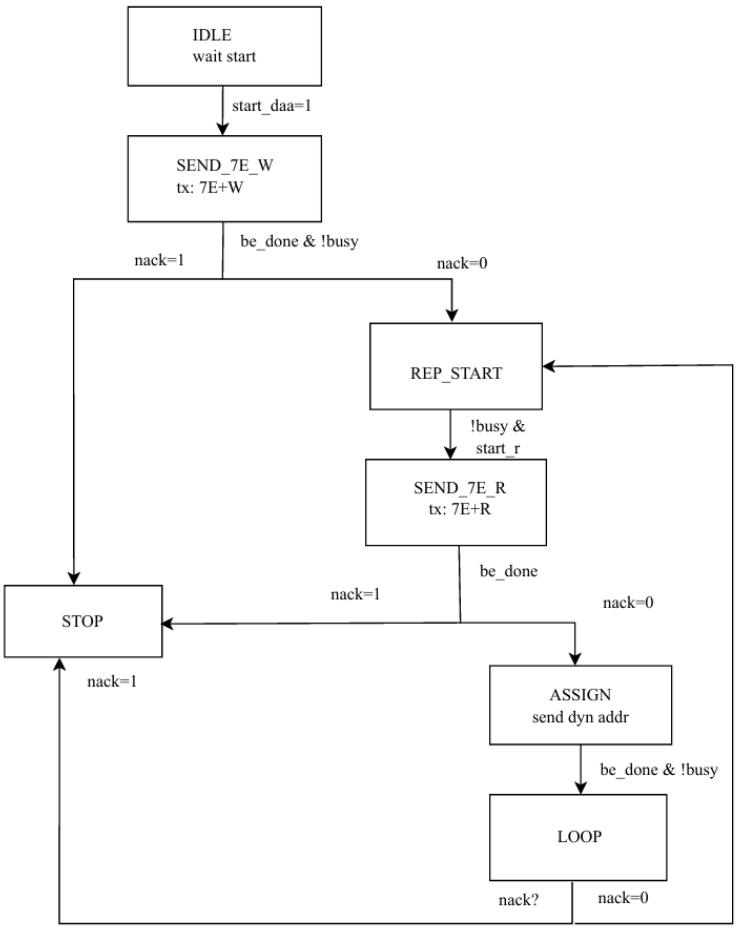

Figure 6.2: Flow chart of DAA FSM

6.2.1 State Diagram

Based on the implemented RTL, the DAA FSM consists of the following states:

- IDLE
- SEND_7E_W
- SEND_ENTDAA
- REP_START
- SEND_7E_R
- ARB_BITS
- ASSIGN
- LOOP
- STOP

The FSM iteratively performs arbitration, PID capture, and dynamic address assignment until no additional unaddressed targets respond.

6.2.2 Arbitration and PID Handling

The DAA FSM performs PID arbitration and capture using bit-level SDA sampling under Bit Engine control.

Arbitration Phase (ARB_BITS):

- After issuing ENTDAA and a repeated START, the controller broadcasts 7'h7E + R
- Unaddressed targets participate in arbitration by driving SDA
- The controller releases SDA and samples the bus on SCL rising edges
- Wired-AND behavior ensures only one target wins arbitration

PID Handling:

- PID, BCR, and DCR fields are captured serially into a shift register
- A fixed 64-bit arbitration window is used for {PID[47:0], BCR[7:0], DCR[7:0]}
- Upon completion, the captured values are latched into internal registers
- The information is made available to the Device Table for storage

Dynamic Address Assignment:

- After successful arbitration, the controller assigns a dynamic address with parity
- The assigned address is acknowledged by the target
- The Device Table write-enable (dt_en) is asserted to record the new device
- The FSM increments the dynamic address and loops to detect additional targets

If no target responds or a NACK is detected, the FSM terminates the DAA sequence.

6.3 FSM Synchronization and Arbitration

The SDR FSM and DAA FSM are mutually exclusive in operation. At any given time, only one FSM is permitted to control the Bit Engine and I3C bus.

FSM synchronization is achieved through:

- Command Control logic that explicitly selects SDR or DAA operation
- Busy signaling from the Bit Engine to serialize bus access
- Deterministic FSM activation based on command type

FSM priority is defined as follows:

- The DAA FSM is activated only upon an explicit Dynamic Address Assignment command
- The SDR FSM handles all standard SDR read/write and CCC transactions

This architecture prevents contention on the I3C bus, guarantees protocol correctness, and simplifies verification by ensuring single ownership of the Bit Engine and bus signals at all times.

Chapter 7

Bit Engine Design

The Bit Engine is the lowest-level protocol block in the I3C Controller IP. It is responsible for generating all I3C bus timing, driving and sampling SDA, and ensuring protocol-compliant behavior for every bit transferred on the bus. Higher-level FSMs rely on the Bit Engine to abstract bit-level signaling and timing details.

7.1 Purpose of Bit Engine

The primary purpose of the Bit Engine is to provide a centralized and reusable mechanism for controlling the physical I3C bus signals. By isolating bit-level behavior into a dedicated block, the overall design achieves improved modularity, timing correctness, and ease of verification.

The Bit Engine:

- Generates protocol-compliant SCL waveforms using SCL Generator Module
- Controls SDA drive and release behavior
- Samples SDA during read and ACK phases
- Implements START and STOP conditions
- Provides handshake signaling to higher-level FSMs

All I3C protocol timing requirements are enforced within the Bit Engine, allowing FSMs to operate at a higher level of abstraction.

7.2 Bit-Level Operations

The Bit Engine supports a set of fundamental bit-level operations that are composed by the FSMs to implement complete I3C transactions.

7.2.1 START and STOP Generation

- **START Condition**
  A START condition is generated by transitioning SDA from high to low while SCL is high. The Bit Engine ensures proper setup and hold timing around the SCL high phase before initiating the transition.
- **STOP Condition**
  A STOP condition is generated by transitioning SDA from low to high while SCL is high. The Bit Engine releases SDA only after satisfying the minimum timing requirements.

START and STOP operations are explicitly requested by the FSMs and acknowledged upon completion.

7.2.2 Bit Transmit Operation

During a bit transmit operation:

- The Bit Engine drives SDA with the requested bit value while SCL is low
- SCL is driven high to allow the target to sample the data bit
- SDA remains stable during the SCL high period
- SCL is driven low to complete the bit cycle

The FSM provides the bit value to transmit, and the Bit Engine reports completion once the full SCL high–low cycle is complete.

7.2.3 Bit Receive Operation

During a bit receive operation:

- The Bit Engine releases SDA to allow the target device to drive the bus
- SDA is sampled during the SCL high period
- The sampled bit value is captured and forwarded to the requesting FSM

The Bit Engine guarantees that sampling occurs within the valid data window defined by the I3C specification.

7.2.4 ACK / NACK Handling

ACK and NACK phases are handled as specialized bit-level operations.

- **ACK Detection**
  During write operations, the Bit Engine samples SDA during the ACK bit time to detect acknowledgment from the target.
- **ACK Generation**
  During read operations, the Bit Engine drives SDA low to generate an ACK or releases SDA to generate a NACK, depending on FSM control.

ACK/NACK results are reported back to the FSM to determine subsequent state transitions.

7.3 Bit Engine Interface Signals

The Bit Engine interfaces with higher-level FSMs using a request–acknowledge handshake model.

Key interface signals include:

- Operation request (START, STOP, transmit bit, receive bit)
- Transmit data bit input
- Received data bit output
- Operation completion indication
- ACK/NACK status indication

This interface ensures that FSMs progress only after the Bit Engine completes the requested operation, preventing timing violations and race conditions.

7.4 Timing Control and SCL Generation

The Bit Engine generates the I3C SCL signal using programmable timing parameters derived from the system clock.

- SCL frequency is configured via register-programmable dividers
- Separate control of SCL high and low periods ensures duty-cycle compliance
- Timing parameters meet SDR mode setup and hold requirements

All SCL transitions are synchronized to the system clock, and no combinational logic directly drives the I3C bus signals. This ensures synthesizability and predictable timing behavior across FPGA and ASIC implementations.

Chapter 8

FIFO Architecture

This chapter describes the FIFO-based data buffering architecture used in the I3C Controller IP. The FIFO structures decouple system-side register access from I3C bus timing, enabling reliable data transfer without stalling protocol execution.

Separate FIFOs are implemented for transmit (TX) and receive (RX) paths to allow independent flow control.

8.1 TX FIFO

The TX FIFO buffers data bytes that are to be transmitted on the I3C bus during write transactions and write-type CCC operations.

8.1.1 FIFO Depth and Width

- The TX FIFO stores data in byte-wide (8-bit) entries
- FIFO depth is fixed and implementation-defined
- Depth is chosen to support burst transfers without frequent software intervention

The FIFO operates synchronously to the system clock and uses internal read and write pointers for tracking occupancy.

8.1.2 Write Operation

- Software writes data into the TX FIFO via the TX FIFO data register
- Each write operation pushes one byte into the FIFO
- Write operations are permitted only when the FIFO is not full
- FIFO full status is reported through status registers

During active transactions:

- The SDR FSM reads bytes from the TX FIFO
- FIFO reads occur at byte boundaries aligned with protocol data phases
- Data availability is verified before initiating transmission

This mechanism ensures continuous data flow during I3C write operations.

8.1.3 Underflow and Overflow Handling

- **Underflow**
  Occurs if the FSM attempts to read data when the TX FIFO is empty. Underflow conditions cause the transaction to terminate and an error status to be reported.

- **Overflow**
  Occurs if software attempts to write data when the TX FIFO is full. Overflow writes are ignored, and a status flag may be asserted.

Underflow and overflow protections prevent invalid bus behavior and data corruption.

8.2 RX FIFO

The RX FIFO buffers data bytes received from the I3C bus during read transactions and read-type CCC operations.

8.2.1 Data Capture Mechanism

- Incoming data bits are sampled by the Bit Engine
- Bytes are assembled and forwarded to the SDR FSM
- Completed bytes are written into the RX FIFO

FIFO writes are performed only when space is available, ensuring data integrity during high-speed transfers.

8.2.2 Read Operation

- Software reads data from the RX FIFO via the RX FIFO data register
- Each read operation pops one byte from the FIFO
- Read operations are permitted only when the FIFO is not empty
- FIFO empty status is reported through status registers

The RX FIFO allows software to retrieve received data asynchronously after transaction completion.

8.2.3 Status Signaling

The RX FIFO provides status indications including:

- FIFO empty
- FIFO full
- Overflow condition

Overflow may occur if incoming data is received faster than software can consume it. In such cases, additional received bytes may be discarded, and an error condition is reported.

Chapter 9

Clocking, Reset, and Timing

This chapter describes the clocking strategy, reset behavior, and timing characteristics of the I3C Controller IP. The design ensures deterministic operation, protocol-compliant timing, and compatibility with standard SoC clocking and reset methodologies.

9.1 System Clock Requirements

The I3C Controller IP operates entirely within a single synchronous clock domain driven by the system clock.

- All internal logic, including FSMs, FIFOs, counters, and the Bit Engine, is clocked by the system clock
- The system clock frequency must be sufficiently higher than the maximum programmed I3C SCL frequency
- No asynchronous clock domain crossings exist within the IP

The minimum system clock frequency is implementation-dependent and must allow accurate generation of SCL high and low periods while meeting setup and hold requirements.

9.2 Programmable SCL Frequency

The I3C SCL signal is generated internally by the Bit Engine using programmable clock dividers derived from the system clock.

- SCL frequency is configurable through dedicated configuration registers
- Separate timing control for SCL high and low phases may be supported
- Duty cycle is maintained within SDR specification limits

SCL generation logic ensures:

- Glitch-free SCL transitions
- Stable SCL frequency during active transactions
- Immediate effect of new SCL settings only when the controller is idle

This approach prevents timing violations during ongoing bus activity.

9.3 Reset Behavior

The I3C Controller IP uses a global active-low reset to initialize the design.

On reset assertion:

- All FSMs enter their IDLE states
- FIFO contents are cleared and pointers reset
- Control and status registers are set to default values

- SDA is released and SCL is driven to an idle state

On reset deassertion:

- The controller remains idle
- No I3C bus activity occurs until explicitly initiated by software

The reset strategy ensures safe startup and recovery from error conditions without causing unintended bus transactions.

9.4 Timing Compliance (SDR Mode)

The I3C Controller IP ensures compliance with SDR mode timing requirements defined by the MIPI I3C specification.

Key timing guarantees include:

- SDA setup and hold times relative to SCL edges
- Minimum SCL high and low periods
- Proper START and STOP timing sequences
- Stable data during SCL high phases

Timing compliance is enforced entirely within the Bit Engine, allowing higher-level FSMs to operate independently of timing details. All timing paths are fully synchronous, enabling straightforward static timing analysis and sign-off.

Chapter 10

Error Handling and Status Reporting

This chapter describes the error detection mechanisms and status reporting architecture of the I3C Controller IP. The controller is designed to detect protocol, command, and data-path errors in real time and to report them to software in a deterministic and debuggable manner.

10.1 NACK Detection

NACK detection is a fundamental error detection mechanism in I3C transactions.

- During address and data phases, the Bit Engine samples the SDA line during the ACK/NACK bit time
- A NACK indicates that the target device did not acknowledge the transmitted byte

Upon detecting a NACK:

- The active FSM transitions to an error-handling path
- The current transaction is terminated safely
- A STOP condition is generated
- A NACK error status is latched

NACK detection applies to:

- Address phase
- Data phase during write transactions
- Directed CCC operations

This ensures that invalid or unresponsive targets do not cause indefinite bus activity.

10.2 Address Error Handling

Address-related errors occur when a target device fails to respond during the address phase.

Address error conditions include:

- No ACK received after address transmission
- Invalid address programming detected by command validation logic

When an address error is detected:

- The transaction is aborted
- A STOP condition is issued
- An address error status flag is set

Address errors are reported distinctly from data-phase errors to aid software debugging and recovery.

10.3 FIFO Error Conditions

FIFO-related errors protect against invalid data movement and ensure data integrity.

- **TX FIFO Underflow**
  Occurs when the controller attempts to read data from an empty TX FIFO during a write transaction.
  Underflow causes the transaction to terminate and an error to be reported.
- **TX FIFO Overflow**
  Occurs when software writes data into a full TX FIFO.
  Overflow writes are ignored and a status flag may be asserted.
- **RX FIFO Overflow**
  Occurs when incoming data is received while the RX FIFO is full.
  Additional received data may be discarded and an overflow status flag is set.

FIFO error conditions do not cause unintended bus activity and are reported for software handling.

10.4 Command Error Reporting

Command-related errors indicate invalid or unsuccessful command execution.

Command error conditions include:

- Unsupported command type
- Invalid command parameter combinations
- Command initiation while the controller is busy
- Premature termination due to protocol or FIFO errors

Command error reporting includes:

- Command completion status
- Error cause indication
- Controller return to idle state

All command-related errors are reflected in status registers, allowing software to identify failure reasons and take corrective action.

Chapter 11

RTL Implementation Details

This chapter describes the RTL implementation structure of the I3C Controller IP. The implementation follows a modular, hierarchical design approach, with each module performing a clearly defined function. The RTL organization closely aligns with the architectural description provided in earlier chapters.

11.1 Module Hierarchy

The I3C Controller IP RTL is organized into the following key modules:

- **parameter.v**
  This file defines all global parameters, register offsets, bit-field positions, and Common Command Code (CCC) values.
  It serves as a centralized definition file to ensure consistency across RTL modules and verification environments.
- **I3C_Controller_Top**
  The top-level integration module that instantiates and connects all functional blocks.
  It exposes the external interfaces, including clock, reset, register interface signals, and I3C bus signals (SCL, SDA).
- **Register Module**
  Implements the memory-mapped register set of the controller.
  Responsibilities include:
  - Register read and write handling
  - Storage of command, address, CCC, and configuration fields
  - Providing control and status signals to other modules
- **Command Control Module**
  Decodes programmed command information from the register module and initiates transaction execution.
  It determines whether a command should be handled by the SDR FSM or the DAA FSM and ensures that only one transaction is active at any given time.
- **SDR FSM Module**
  Implements the control logic for all SDR-based transactions, including private read/write operations and CCC handling.
  This FSM sequences protocol phases and interacts with the Bit Engine for bit-level execution.
- **DAA FSM Module**
  Implements the Dynamic Address Assignment (ENTDAA) flow.
  It handles PID arbitration, PID reception, and dynamic address assignment, interacting with the Bit Engine and the Device Table.
- **Bit Engine Module**
  Responsible for all bit-level protocol operations, including:
  - START and STOP generation
  - Bit transmit and receive
  - ACK/NACK handling

- SDA drive and sampling

The Bit Engine abstracts all timing-sensitive bus behavior from higher-level FSMs.

- **SCL Generator Module**
  Generates the I3C SCL signal based on programmable timing parameters.
  It provides clock enable and phase control signals to the Bit Engine, ensuring protocol-compliant SCL timing.
- **TX FIFO Module**
  Buffers outgoing data bytes written by software before transmission on the I3C bus.
- **RX FIFO Module**
  Buffers incoming data bytes received from the I3C bus for later retrieval by software.
- **Device Table Module**
  Stores information related to dynamically addressed I3C target devices, such as assigned dynamic addresses and associated PID data.
  This module is primarily used during and after the DAA process.

This modular hierarchy enables clear separation of responsibilities, simplified verification, and straightforward future extensibility.

11.2 Parameterization

The RTL design uses compile-time parameters to configure structural aspects of the implementation.

Parameterization includes:

- FIFO depth and width
- SCL timing divider values
- Maximum supported transfer length

All parameters are resolved at synthesis time. Functional behavior is independent of runtime parameter changes, ensuring deterministic operation and ease of verification.

11.3 Coding Guidelines

The RTL follows standard industry coding guidelines:

- Fully synchronous design using a single system clock
- Active-low synchronous reset
- Non-blocking assignments for sequential logic
- Blocking assignments for combinational logic
- Explicit default assignments to avoid latch inference
- Clear separation of combinational and sequential blocks
- Consistent naming conventions across modules

FSMs are implemented using explicit state encoding for clarity and debug visibility.

11.4 Synthesizability Considerations

The I3C Controller IP RTL is fully synthesizable for FPGA and ASIC targets.

Key considerations include:

- No use of unsynthesizable constructs such as delays or real-time system tasks
- No gated clocks; all clocking is centralized
- Registered control of SCL and SDA outputs
- Clean reset behavior for all sequential elements
- No combinational feedback paths

The design supports clean static timing analysis and is suitable for synthesis, place-and-route, and silicon implementation flows.

Chapter 12

Verification Readiness

This chapter describes the verification-oriented aspects of the I3C Controller IP. The design is structured to support efficient functional verification, assertion-based verification, and debug, enabling rapid validation and issue isolation.

12.1 Design for Verification (DFV)

The I3C Controller IP is developed with Design for Verification (DFV) principles to ensure that all functional behavior is observable, controllable, and testable.

Key DFV considerations include:

- Clear separation between control logic, protocol FSMs, and data-path logic
- Deterministic FSM behavior with well-defined state transitions
- Centralized bit-level control within the Bit Engine
- FIFO-based buffering to isolate bus timing from software interaction

The command-driven execution model allows verification environments to easily stimulate different transaction types and observe outcomes through status registers and FIFO contents.

12.2 Assertion Strategy

Assertion-based verification is supported at multiple abstraction levels.

Typical assertion categories include:

- **Protocol Assertions**
  Verify correct sequencing of START, address, data, ACK/NACK, and STOP phases.
- **FSM Assertions**
  Ensure valid state transitions and prevent illegal state encodings.
- **FIFO Assertions**
  Check for overflow, underflow, and correct pointer behavior.
- **Timing Assertions**
  Validate SDA stability during SCL high periods and correct START/STOP timing.

Assertions may be embedded in RTL or implemented in the verification environment. The modular structure of the design enables targeted assertion placement with minimal cross-module dependencies.

12.3 RTL Observability and Debug Signals

The RTL provides sufficient internal visibility to support debug and waveform analysis.

Observability features include:

- FSM state outputs for SDR FSM and DAA FSM
- Bit Engine operation status and completion signals

- FIFO level and status indicators
- Command execution status and error flags

These signals can be exposed conditionally for simulation or connected to debug interfaces in silicon, if required. The design avoids hidden or opaque behavior, enabling efficient root-cause analysis during verification and bring-up.

Chapter 13

Performance and Limitations

This chapter describes the performance characteristics and known limitations of the I3C Controller IP Version 1.0. The information provided here helps system integrators, firmware developers, and verification teams understand expected operational bounds and trade-offs.

13.1 Maximum Supported Transfer Size

The maximum supported transfer size for a single I3C transaction is determined by the programmable transfer length register and the available FIFO depth.

Key considerations include:

- Transfer length is specified in bytes via a dedicated register
- TX and RX FIFO depths limit the maximum continuous data transfer without software intervention
- Large transfers may require software to manage FIFO refills or draining

There is no fixed architectural limit imposed by the protocol FSMs beyond configured register and FIFO constraints.

13.2 Throughput Considerations

The achievable data throughput depends on multiple factors:

- Programmed I3C SCL frequency
- Transaction type (read, write, CCC)
- ACK/NACK overhead and protocol framing
- FIFO availability and software interaction latency

In SDR mode, data throughput is limited by single data transfers per SCL cycle. The Bit Engine ensures protocol-compliant timing, prioritizing correctness over maximum throughput.

Optimal throughput is achieved when:

- FIFO depths are sufficient to avoid underflow or overflow
- Software programs transactions efficiently
- SCL frequency is configured near the maximum supported value

13.3 Known Limitations (v1.0)

The following limitations apply to I3C Controller IP Version 1.0:

- SDR mode operation only
- No support for HDR transfer modes
- No In-Band Interrupt (IBI) support

- Controller (master) mode only
- No multi-controller arbitration
- No Hot-Join support
- Limited parameterization to simplify verification

These limitations are intentional and align with the scope of Version 1.0. Future versions may extend functionality to address these areas.

Chapter 14

Future Enhancements

This chapter outlines potential enhancements planned for future versions of the I3C Controller IP. These enhancements build upon the existing modular architecture and are intended to extend functionality, performance, and protocol coverage while maintaining backward compatibility.

14.1 HDR Mode Support

Future versions of the I3C Controller IP may support High Data Rate (HDR) modes defined by the MIPI I3C specification.

Planned enhancements include:

- Support for HDR-DDR and other HDR transfer modes
- Extended Bit Engine capabilities for multi-edge data sampling
- Enhanced timing generation and phase control
- HDR-specific FSM extensions

HDR support will significantly increase data throughput while maintaining I3C protocol compatibility.

14.2 IBI Support

In-Band Interrupt (IBI) support enables target devices to asynchronously notify the controller of events.

Potential enhancements include:

- IBI detection and arbitration handling
- IBI queueing and prioritization
- Software-visible IBI status and interrupt reporting

IBI support will enable more efficient event-driven communication in sensor-heavy systems.

14.3 Slave Mode Support

Adding target (slave) mode capability will allow the IP to operate as an I3C device rather than solely as a controller.

Possible features include:

- Dynamic and static address response
- Target-side FSMs for SDR and CCC handling
- Bidirectional data buffering
- Support for controller-initiated and target-initiated transactions

Slave mode support would enable broader reuse of the IP in multi-role systems.

14.4 Multi-Controller Support

Multi-controller support enables multiple I3C controllers to share a single bus.

Future enhancements may include:

- Bus arbitration logic
- Controller handoff and priority management
- Collision detection and recovery

Multi-controller support will enable more complex system topologies and improve system scalability.

Chapter 15

Conclusion

This document has presented the complete architectural, functional, and implementation specification of the I3C Controller IP Version 1.0. The IP is designed as a modular, synthesizable, and verification-ready implementation of an I3C controller supporting SDR mode operation, Dynamic Address Assignment, and Common Command Code handling.

The architecture cleanly separates register programming, command control, protocol FSMs, bit-level timing, and data buffering. This structured approach simplifies integration, verification, and future extensibility. All supported features are implemented in compliance with the MIPI I3C specification for SDR controller operation.

Version 1.0 intentionally focuses on a stable and verifiable feature set, excluding advanced protocol capabilities such as HDR modes, In-Band Interrupts, and slave mode operation. These exclusions allow the IP to serve as a robust foundation for early integration, bring-up, and system validation.

The design is fully synthesizable, follows industry-standard RTL coding practices, and includes sufficient observability to support efficient verification and debug. Clear documentation of performance characteristics, limitations, and future enhancements ensures transparency and sets a well-defined roadmap for subsequent versions.

Overall, the I3C Controller IP v1.0 represents a solid, production-ready baseline that can be extended in future releases to support advanced I3C features and broader system requirements.

Appendix A
Signal List

This appendix lists the primary external and internal signals used in the I3C Controller IP. The signals are grouped by functional interface for clarity and ease of reference.

Table A.1: Top-Level Module: I3C_TOP

Signal Name	Direction	Width	Description
clk	Input	1	System clock
rst_n	Input	1	Active-low asynchronous reset

Table A.1.2: CPU / Register Interface

Signal Name	Direction	Width	Description
wr_en	Input	1	Register write enable
rd_en	Input	1	Register read enable
addrs	Input	7	Register address
w_reg_data	Input	32	Register write data
w_data	Input	8	FIFO write data
rd_data	Output	32	Register read data
r_data	Output	8	FIFO read data

Table A.1.3: I3C Bus Interface Signals

Signal Name	Direction	Width	Description
scl_i	Input	1	Sampled SCL input
scl_o	Output	1	Driven SCL output
sda_i	Input	1	Sampled SDA input
sda_o	Output	1	Driven SDA output
sda_oe	Output	1	SDA output enable (tri-state control)

A.2 Register Interface Module: i3c_reg_interface

Table A.2.1: Control and Status Signals

Signal Name	Direction	Width	Description
cmd_start	Output	1	Initiates a new command
cmd_type	Output	2	Command type (SDR / CCC / DAA)
cmd_addr	Output	7	Target address
cmd_ccc	Output	8	CCC opcode
cmd_len	Output	8	Transfer length
cmd_dir	Output	1	Transfer direction (0=Write, 1=Read)
SDR_DONE	Input	1	SDR transaction complete
DAA_DONE	Input	1	DAA transaction complete
SDR_BUSY	Input	1	SDR FSM busy
DAA_BUSY	Input	1	DAA FSM busy
SDR_ERROR	Input	1	SDR error flag

DAA_ERROR	Input	1	DAA error flag

A.3 FIFO Interface Signals

Table A.3.1: TX FIFO (i3c_Tx_FIFO)

Signal Name	Direction	Width	Description
Tx_wr_en	Input	1	Write enable
Tx_wdata	Input	8	Data to be transmitted
tx_valid	Output	1	TX FIFO data valid
tx_ready	Input	1	Read enable from FSM
tx_rdata	Output	8	Data read from TX FIFO
tx_full	Output	1	FIFO full indicator
tx_empty	Output	1	FIFO empty indicator

Table A.3.2: RX FIFO (i3c_Rx_FIFO)

Signal Name	Direction	Width	Description
rx_valid	Input	1	Write enable from FSM
rx_wdata	Input	8	Received data
Rx_rd_en	Input	1	Read enable
rx_rdata	Output	8	Data read from RX FIFO
rx_full	Output	1	FIFO full
rx_empty	Output	1	FIFO empty

Table A.4: SDR FSM Module: i3c_sdr_fsm

Signal Name	Direction	Width	Description
start	Input	1	Start SDR transaction
addr	Input	7	Target address
dir	Input	1	Read / Write select
len	Input	8	Number of bytes
busy	Output	1	FSM busy
done	Output	1	Transaction complete
error	Output	1	Error detected
push_pull	Output	1	Push-pull enable for SDA
be_valid	Output	1	Bit Engine request
be_rd_wr	Output	1	Bit Engine direction
be_tx_data	Output	8	Byte to Bit Engine

be_rx_data	Input	8	Byte from Bit Engine
be_busy	Input	1	Bit Engine busy
be_done	Input	1	Bit Engine done
be_nack	Input	1	NACK detected

Table A.5: DAA FSM Module: i3c_daa_fsm

Signal Name	Direction	Width	Description
start_daa	Input	1	Start ENTDAA sequence
daa_done	Output	1	DAA completed
valid	Output	1	Bit Engine request
rd_wr	Output	1	Read/Write control
tx_data	Output	8	Data to Bit Engine
arb_mode	Output	1	Arbitration mode enable
pid	Output	48	Provisional ID
bcr	Output	8	Bus Characteristics Register
dcr	Output	8	Device Characteristics Register
dyn_addr	Output	7	Assigned dynamic address
dt_en	Output	1	Device table write enable

Table A.6: Bit Engine Module: i3c_bit_engine

Signal Name	Direction	Width	Description
valid	Input	1	Start bit-level operation
rd_wr	Input	1	Read / Write
tx_data	Input	8	Data to transmit
rx_data	Output	8	Data received
busy	Output	1	Bit Engine busy
nack	Output	1	NACK detected
start_done	Output	1	START condition complete
stop_done	Output	1	STOP condition complete
arb_mode	Input	1	Arbitration mode
push_pull	Input	1	Push-pull enable

Table A.7: Device Table Module: i3c_device_table

Signal Name	Direction	Width	Description
wr_en	Input	1	Write enable
wr_data	Input	71	{PID, BCR, DCR, Dynamic Address}
rd_en	Input	1	Read enable
rd_data	Output	7	Dynamic address

rd_valid	Output	1	Read valid
is_i3c	Output	1	Device type indicator

Table A.8: SCL Generator Module: i3c_scl_gen

Signal Name	Direction	Width	Description
scl_start	Input	1	Start SCL generation
scl_stop	Input	1	Stop SCL generation
scl_hz_cfg	Input	32	Requested SCL frequency
scl_o	Output	1	Driven SCL
scl_oe	Output	1	SCL output enable
push_pull	Input	1	Push-pull control

Appendix B
FSM State Tables

This appendix documents the state encodings and transitions for the major protocol FSMs implemented in the I3C Controller IP.

Table B.1: SDR FSM State Table

State Name	Description	Transition Conditions
IDLE	Idle state, waits for start	start → SEND_ADDR
SEND_ADDR	Send address byte	!be_busy → WAIT_ADDR
WAIT_ADDR	Wait for address ACK	ACK → READ/WRITE, NACK → ERROR
SEND_WR	Send write data	tx_valid → WAIT_WR
WAIT_WR	Wait for write completion	Last byte → STOP
SEND_RD	Initiate read	!be_busy → WAIT_RD
WAIT_RD	Receive data	Last byte → STOP
STOP	Generate STOP	→ IDLE
ERROR	Error handling	→ IDLE

Table B.2: DAA FSM State Table

State Name	Description	Transition Conditions
IDLE	Wait for DAA command	start_daa → SEND_7E_W
SEND_7E_W	Broadcast 7E + W	ACK → SEND_ENTDAA
SEND_ENTDAA	Send ENTDAA CCC	ACK → REP_START
REP_START	Generate repeated START	→ SEND_7E_R
SEND_7E_R	Broadcast 7E + R	ACK → ARB_BITS
ARB_BITS	PID/BCR/DCR arbitration	bit_cnt=0 → ASSIGN
ASSIGN	Assign dynamic address	→ LOOP
LOOP	Store device, increment address	→ REP_START or STOP
STOP	End DAA	→ IDLE

Appendix C

Timing Diagrams

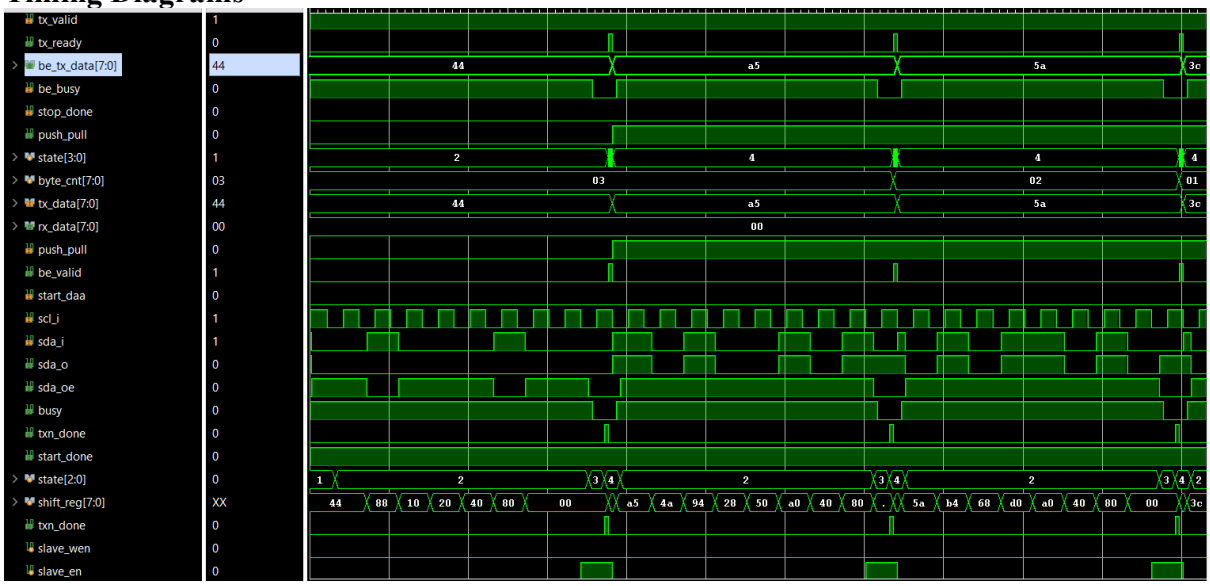

Figure C.1: SDR Address Phase and Write Transaction Timing Diagram

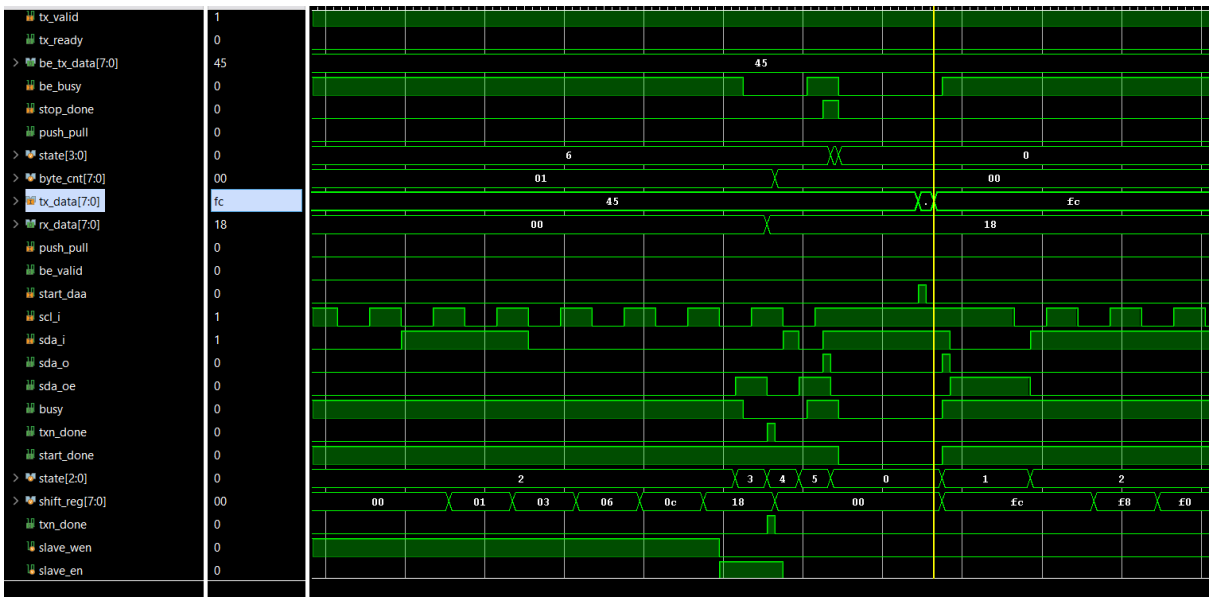

Figure C.2: SDR Read Transaction Timing Diagram

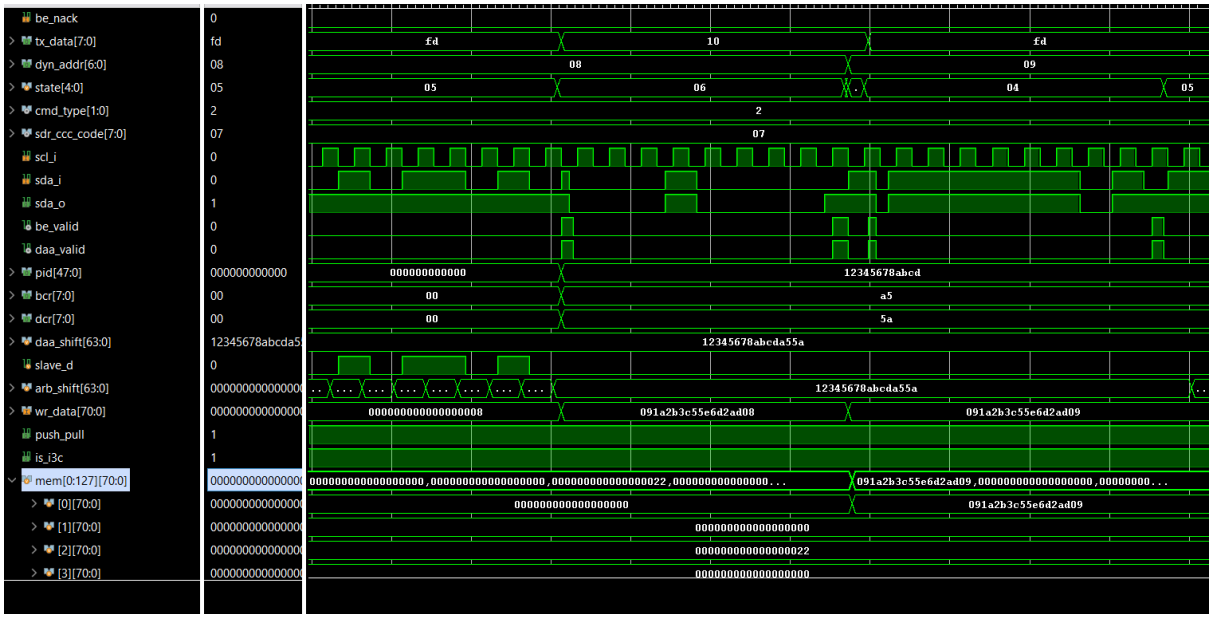

Figure C.3: DAA Arbitration and Dynamic Address Assignment with Device Table Update

Appendix D
References

The following documents and standards were referenced during the design and documentation of the I3C Controller IP:

1. **MIPI Alliance**
   MIPI I3C Basic Specification, Latest Revision
2. **MIPI Alliance**
   MIPI I3C Controller and Target Interface Specification
3. **MIPI Alliance**
   I3C Common Command Codes (CCC) Definition
4. **IEEE**
   IEEE Std 1800™ – SystemVerilog Language Reference Manual
5. **Internal Design Artifacts**
   - I3C Controller IP RTL source code
   - Internal verification testbenches and simulation results